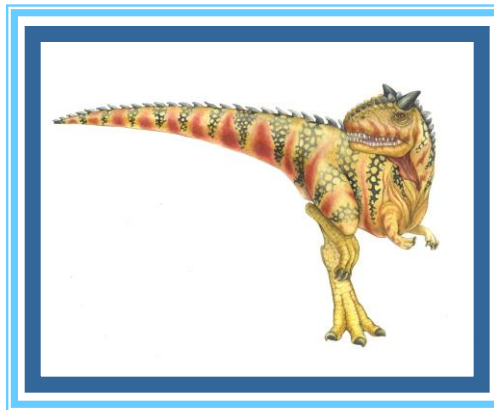


Chapter 3: Processes

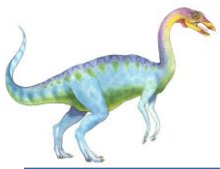




Outline

- Examples of IPC Systems
- Communication in Client-Server Systems





Producer-Consumer: Message Passing

- Producer

```
message next_produced;  
while (true) {  
    /* produce an item in next_produced */  
  
    send(next_produced);  
}
```

- Consumer

```
message next_consumed;  
while (true) {  
    receive(next_consumed)  
  
    /* consume the item in next_consumed */  
}
```

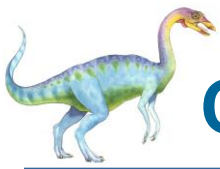




Buffering

- Queue of messages attached to the link.
- Implemented in one of three ways
 1. Zero capacity – no messages are queued on a link.
Sender must wait for receiver (rendezvous)
 2. Bounded capacity – finite length of n messages
Sender must wait if link full
 3. Unbounded capacity – infinite length
Sender never waits





Communications in Client-Server Systems

- Sockets
- Remote Procedure Calls





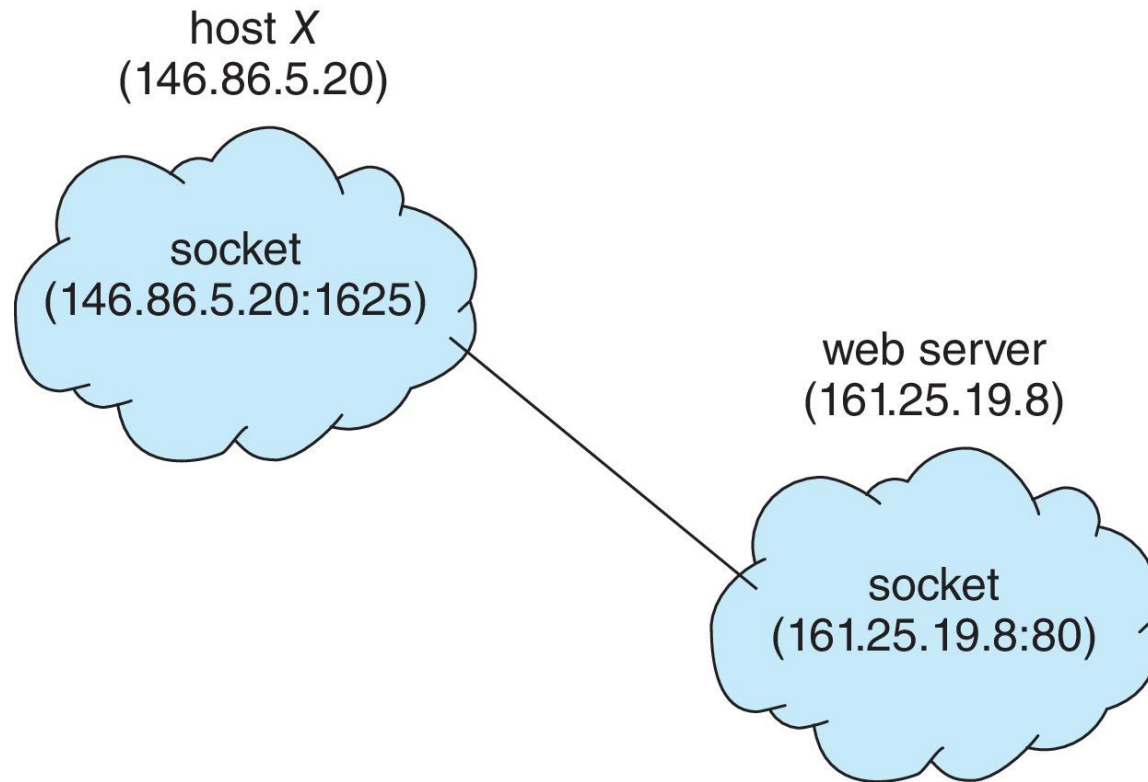
Sockets

- A **socket** is defined as an endpoint for communication
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets
- All ports below 1024 are **well known**, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running





Socket Communication





Sockets in Java

- Three types of sockets
 - **Connection-oriented (TCP)**
 - **Connectionless (UDP)**
 - **MulticastSocket** class— data can be sent to multiple recipients
- Consider this “Date” server in Java:

```
import java.net.*;
import java.io.*;

public class DateServer
{
    public static void main(String[] args) {
        try {
            ServerSocket sock = new ServerSocket(6013);

            /* now listen for connections */
            while (true) {
                Socket client = sock.accept();

                PrintWriter pout = new
                    PrintWriter(client.getOutputStream(), true);

                /* write the Date to the socket */
                pout.println(new java.util.Date().toString());

                /* close the socket and resume */
                /* listening for connections */
                client.close();
            }
        }
        catch (IOException ioe) {
            System.err.println(ioe);
        }
    }
}
```





Sockets in Java

The equivalent Date client

```
import java.net.*;
import java.io.*;

public class DateClient
{
    public static void main(String[] args) {
        try {
            /* make connection to server socket */
            Socket sock = new Socket("127.0.0.1",6013);

            InputStream in = sock.getInputStream();
            BufferedReader bin = new
                BufferedReader(new InputStreamReader(in));

            /* read the date from the socket */
            String line;
            while ( (line = bin.readLine()) != null)
                System.out.println(line);

            /* close the socket connection*/
            sock.close();
        }
        catch (IOException ioe) {
            System.err.println(ioe);
        }
    }
}
```





Dezavantaje message passing

- modelul de programare uzual pt sisteme centralizate nu functioneaza pt sisteme distribuite
 - nu exista reprezentarea locala a resurselor distribuite => rutinele/functiile locale nu le pot accesa
 - trebuie modificate programele obisnuite pt a gestiona comunicatia la distanta (problema de I/O message passing)
- apar dificultati de programare
 - un mesaj e doar un sir de biti, programele opereaza cu tipuri de date complexe
 - e necesara pastrarea tipurilor de date la transmisia datelor peste retea
 - arhitecturile de calcul conectate in retea pot fi eterogene, cu reprezentare diferita a tipurilor de date
 - ex: intregi reprezentati in format little (Intel) respective big (Sparc) endian





Remote Procedure Calls (RPC)

- Birell & Nelson, 1984
 - calculul transcede limitelor masinii locale apeland proceduri aflate pe masini la distanta
 - avantaj: model de programare centralizat, nu exista diferente intre apelurile locale si cele distante (ideal)
 - natura distribuita a sistemului impiedica totusi o identificare a celor doua tipuri de apeluri
 - procedura apelanta si cea apelata nu se afla in acelasi spatiu de adresa
- => pasarea parametrilor si intoarcerea rezultatului pot fi complicate
- defectarea (“crash”) oricareia dintre cele doua masini fizice implicate creeaza probleme suplimentare, deopotriva tehnice si de semantica a apelului
 - totusi modelul e popular pe arhitecturi NORMA
 - Java RMI, Web services (SOAP)





Functionarea RPC

- programul client se linkediteaza cu asa-numite rutine *stub*
 - invocate in locul procedurii propriu-zise aflate pe masina de la distanta
- invocarea stub-ului client determina urmatorul sir de actiuni:
 - (1) stub-ul converteste argumentele procedurii intr-un format independent de masina si le impacheteaza intr-un mesaj (*marshalling*)
 - (2) mesajul este livrat protocolului de transport din kernel (eg, TCP/UDP)
 - (3) protocolul de transport determina transmisia mesajului la distanta catre o rutina stub din server
 - despacheteaza argumentele procedurii (si identificatorul ei) si le converteste la reprezentarea masinii locale (*unmarshalling*)
 - (4) stub-ul server apeleaza procedura propriu-zisa





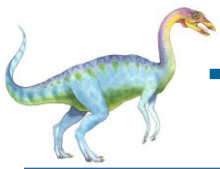
Functionarea RPC (cont.)

- invocarea stub-ului client determina urmatorul sir de actiuni:

....

- (5) rezultatele procedurii sunt supuse procedurii de marshalling intr-un mesaj de raspuns
- (6) mesajul de raspuns e trimis inapoi clientului
- (7) stub-ul client despacheteaza mesajul de raspuns si converteste datele la formatul local
- (8) stub-ul client intoarce valoarea rezultatului RPC procedurii apelante





Transmiterea parametrilor in RPC

- *call by value*
 - se paseaza valoarea argumentului pe stiva
 - argumentul modificabil local in procedura, dar modificarile se pierd la return
 - in principiu, potrivit pt RPC
- *call by reference*
 - doar un pointer (adresa) catre date e pasat pe stiva
 - modificarile datelor facute prin indirectare supravietuiesc apelului (se vad la nivelul apelantului)
 - nu are sens pt RPC unde clientul si serverul nu partajeaza spatiile de adresa
- *copy/restore*
 - varianta asimilabila *call by value*
 - *call by value* dar la retur se copiaza inapoi valoarea argumentului suprascriindu-se valoarea apelantului
 - folosit in RPC





Reprezentarea datelor

- pana si tipurile de baza pot avea reprezentari diferite pe masini diferite (v. intregi)
- functionarea in medii eterogene parametrii necesita o reprezentare standard folosita de stub-uri
 - ex: XDR (eXternal Data Representation), ASN.1 (Abstract Syntax Notation)
 - transformari de nivel de prezentare a datelor in modelul OSI (nivelul 6, imediat inainte de nivelul de aplicatie)
 - tot aici se pot opera transformari ale datelor de tip criptare (inclusiv in RPC)
- situatii clasice
 - intregi reprezentati little/big endian
 - string-uri codificate ASCII/EBCDIC/Unicode
- Obs: la intregi transformarea e simpla (se inverseaza octetii), dar la string-uri problema e mai complicata => nevoia de format standard
- optimizare: intre masini cu arhitecturi compatibile, convertirea e inutila, se recurge la notificarea tipului de format in mesaj





Protocolul de transport

- initial era prevazut un protocol dedicat
- UDP
 - fara overhead de setare a conexiunii
 - clientul trebuie sa trateze retransmisia pachetelor, erorile de transmisie, secventierea pachetelor
 - totusi, e protocolul preferat pe LAN unde pierderea de pachete si erorile sunt rare
 - oricum, mai putin performant decat un protocol dedicat (care n-ar face asamblarea/dezasamblarea fragmentelor, diverse checksum-uri, etc)
- TCP – potrivit pt WAN, unde tratarea automata a retransmisiei, erorilor si a secventierii mesajelor asigurata de TCP e un avantaj
- dimensiunea pachetului de retea folosita
 - overhead-ul RPC e fix, indiferent de marimea datelor trimise
 - pt performanta amortizata buna, se prefera transmisia mai multor date intr-un singur mesaj
 - conteaza mediul de transmisie, v. Ethernet standard vs jumbo frames





Semantica RPC

- diferita de semantica apelurilor locale
 - (1) se pot pierde mesaje
 - (2) se pot duplica mesaje
 - (3) serverul si/sau clientul se pot defecta/opri
- (1) & (2) se rezolva prin retransmisie si folosire nr de secventa
- operatii *idempotente*: efectul executiei multiple este acelasi cu cel al unei singure executii
 - ex. setarea unui bit, citirea primului bloc de date dintr-un fisier
- ex. operatii care nu sunt idempotente: incrementarea unui intreg, adaugarea datelor la sfarsitul unui fisier (operatia de append)





Semantica RPC (cont.)

- *exactly once*
 - procedura de la distanta se executa o singura data
 - greu de asigurat
 - mesaje duplicate, rezolvate prin retransmisie si nr de secventa
 - server crash: rezolvare dificila, serverul se poate defecta
 - *inainte* de servirea cererii
 - *dupa* servirea cererii dar *inainte* de transmitia unui raspuns
 - *dupa* transmitia unui raspuns care se pierde
- *at most once*
 - procedura remote fie nu se executa deloc fie cel mult o data
 - fara un raspuns de la server nu e clar daca operatia s-a executat sau nu
- *at least once*
 - procedura remote se executa cel putin o data (posibil de mai multe ori)
 - semantica potrivita pentru operatii idempotente
 - clientul retransmite cererea pana cand primeste ACK





Defectarea clientului, solutii

- *exterminare*
 - fiecare apel RPC se inregistreaza pe disc intr-un log
 - dupa reboot, daca vin raspunsuri de la server se ignora (“calculul e exterminat”)
- *reincarnare*
 - foloseste conceptul de epoca
 - la reboot clientul anunta prin broadcast o noua epoca tuturor serverelor, care in consecinta termina calculele aferente tuturor epocilor trecute
 - daca apar totusi raspunsuri din epoci trecute, clientul le ignora
- *reincarnare delicata*
 - varianta, serverele notificate la inceputul unei noi epoci termina calculul doar daca nu pot identifica proprietarul lui





Defectarea clientului, solutii (cont.)

- *expirare*
 - fiecare RPC trebuie sa termine intr-un interval fix de timp T
 - dezavantaj: daca nu se poate termina in timp util, trebuie ceruta o noua perioada
 - avantaj: daca dupa crash clientul asteapta o perioada T , atunci stie sigur ca nici un calcul nu mai e activ
- Obs: nicio solutie nu e satisfacatoare (ex: dc. procedura remote care se termina fortat detine locks, resursele respective protejate de locks raman blocate definitiv)





Binding

- procesul prin care clientul determina adresa serverului (host si port)
- (1) director de servicii global
 - serverele isi inregistreaza adresele si tipul de serviciu pe care il ofera in director
 - clientii cauta in director adresa unui server care ofera serviciul dorit
- (2) clientul stie adresa serverului
 - serverul ruleaza propriul director de servicii in care programele server isi inregistreaza serviciile
 - ex: SUN portmapper
- clientii folosesc porturi publice (*well-known ports*) pentru a adresa cereri directorului de servicii





Suport RPC la nivelul limbajului

- RPC e o constructie de nivelul limbajelor de programare
 - compilatorul trebuie informat ca procedura se va executa la distanta si ca apelurile se fac prin stub
 - diferenta de semantica fata de apelurile obisnuite => marcarea explicita a apelurilor RPC la nivel de limbaj prin keywords de tipul *remoteproc*
- solutia alternativa: Interface Definition Language
 - fara extensii de compilator pt suport RPC
 - defineste punctul de interactiune intre client si server ca fiind o *interfata*
 - definita intr-un limbaj special de definire a interfetei IDL
 - specificarea interfetei IDL compilata cu un compilator special care genereaza stub-uri client si server (ex: Sun *rpcgen*)
 - diferite compilatoare IDL pot fi folosite pt a genera stub-uri pt limbaje si conventii de apelare diferite
 - alternativ, exista limbaje care suporta nativ conceptul de interfata, interfata RPC nefiind decat un caz particular (ex: Java RMI)

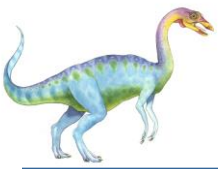




Exemplu de implementare RPC

- Sun RPC are propriul IDL si un compilator specific (*rpcgen*) numit si “generator de stub-uri”
- fiecare masina server ruleaza un director de servicii numit *portmapper*
 - mapeaza identificatorul programului server pe un port de comunicatie
 - programul server e identificat prin nr de program si versiune
- exemplu de specificare a interfetei pt un serviciu simplu de fisiere
 - interfata suporta operatia SFS_READ (operatia cu nr 1)
 - primeste un argument de tip *readargs*, intoarce un rezultat de tip *readres*
 - programul server are numarul 123321 si versiunea 9876
- interfata definita in *sfs.x* iar stub-urile generate cu comanda *rpcgen sfs.x =>*
 - *sfs_client.c* – stub client
 - *sfs_server.c* – stub server
 - *sfs_xdr.c* – rutinele de marshalling/unmarshalling
 - *sfs.h* – interfata in limbaj C





Interfata IDL server de fisiere

```
1 const SFS_MAXDATA = 4096;
2
3 struct readres {
4     int count;
5     opaque data<SFS_MAXDATA>;
6 };
7
8 struct readargs {
9     string file<SFS_MAXDATA>;
10    unsigned offset;
11    unsigned count;
12 };
13
14 program SFS_PROGRAM {
15     version SFS_VERSION {
16         readres SFS_READ(readargs) = 1;
17     } = 9876;
18 } = 123321;
```

Figure 2: Simple File Server interface definition file sfs.x.

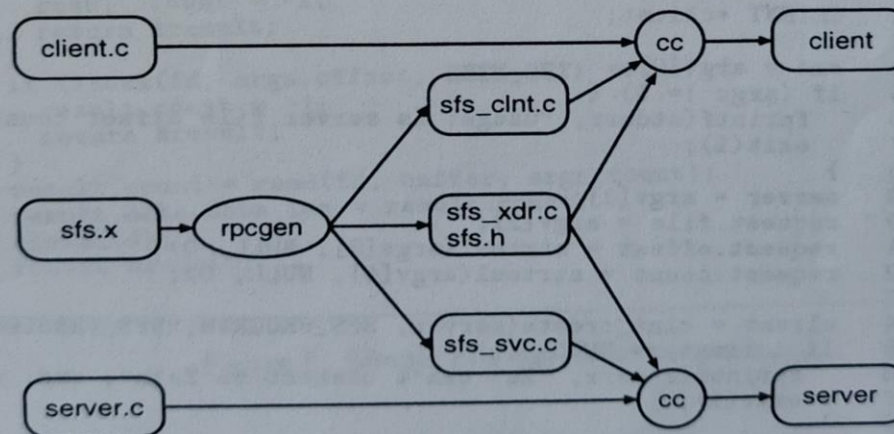


Figure 3: rpcgen operation.





Codul RPC client

- *sfs_client.c*
 - construiește o cerere
 - creează un handle folosit pt. a executa RPC-uri pe server
 - *clnt_create*
 - contactează *portmapper-ul* de pe server
 - cere portul corespunzător perechii (SFS_PROGRAM, SFS_VERSION)
 - creează un socket UDP pt comunicare cu serverul
- apelul RPC = apelul *sfs_read_9786*
 - folosește o structură pt ca inițial *rpcgen-ul* nu accepta decât proceduri cu un singur argument
 - versiunile moderne accepta mai mulți parametri

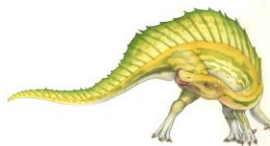


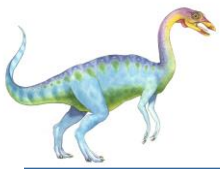


Cod RPC client

```
1 #include <rpc.h>
2 #include "sfs.h"
3
4 char *cmd;
5
6 int
7 main(int argc, char *argv[])
8 {
9     struct readargs request;
10    struct readres *result;
11    char *server;
12    CLIENT *client;
13
14    cmd = argv[0];
15    if (argc != 5) {
16        fprintf(stderr, "usage: %s server file offset count\n", cmd);
17        exit(1);
18    }
19    server = argv[1];
20    request.file = argv[2];
21    request.offset = strtoul(argv[3], NULL, 0);
22    request.count = strtoul(argv[4], NULL, 0);
23
24    client = clnt_create(server, SFS_PROGRAM, SFS_VERSION, "udp");
25    if (client == NULL) {
26        fprintf(stderr, "%s: can't connect to %s\n", cmd, server);
27        exit(1);
28    }
29    result = sfs_read_9876(request, client);
30    if (result == NULL) {
31        fprintf(stderr, "%s: read failed\n", cmd);
32        exit(1);
33    }
34    write(1, result->data.data_val, result->data.data_len);
35    exit(0);
36 }
```

Figure 4: Simple File Server client code.





Codul RPC server

- *sfs_server.c*
 - apeleaza procedura *sfs_read_9786_svc*
 - aceasta deschide fisierul solicitat
 - muta file pointerul la offsetul solicitat
 - citeste nr de octeti solicitat
 - returneaza rezultatul





Cod RPC server

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <rpc.h>
4 #include "sfs.h"
5
6 readres *
7 sfs_read_9876_svc(readargs args, struct svc_req *request)
8 {
9     static readres result;
10    static char buffer[SFS_MAXDATA];
11    int count;
12    int fd;
13
14    result.count = 0;
15    result.data.data_len = 0;
16    result.data.data_val = buffer;
17
18    if ((fd = open(args.file, O_RDONLY)) == -1) {
19        result.count = -1;
20        return &result;
21    }
22    if (lseek(fd, args.offset, SEEK_SET) == -1) {
23        result.count = -1;
24        return &result;
25    }
26    result.count = read(fd, buffer, args.count);
27    result.data.data_len = result.count;
28    close(fd);
29    return &result;
30 }
```

Figure 5: Simple File Server server code.

